Ders 9 Çalışma Özeti

Ders 9 Çalışma Özeti

Genel Konular

- Yığın (stack) yapısının kullanım alanları, avantajları ve dezavantajları.
- Infix ifadeyi prefix ifadeye dönüştürme (infix to prefix).
- Ekstra yığın kullanmadan, bağlı liste ile kurulmuş bir yığını ters çevirme.
- Kuyruk (queue) yapısı: FIFO mantığı, `front` ve `rear` işaretçileri, `enqueue` ve `dequeue`.
- Kuyruğun dizi ve bağlı liste ile gerçeklenmesi.
- Dairesel kuyruk (circular queue): boşluk ve doluluk koşulları, modüler aritmetik.
- Yığın ile kuyruğun karşılaştırılması.
- İki yığınla kuyruk, kuyruk ile yığın gerçekleme soruları.

Hocanın Özellikle Vurguladığı Kısımlar

- Yığın, LIFO (last in first out) çalışır. Gelen sıranın tersine hizmet verir. Bu yapıya uyan problemler yığınla kurgulanmalıdır.
- Infix'ten prefix'e dönüşüm üç adımdır: ifadeyi ters çevir, postfix'e dönüştür, sonucu tekrar ters çevir.
- Ters çevirmede parantezler de yer değiştirir: `(` yerine `)`, `)` yerine `(` yazılır.
- Prefix dönüşümü postfix'ten daha zahmetlidir. Bu yüzden derleyiciler genelde postfix üzerinden çalışır.
- Yığında yalnız en üstteki elemana (`top`) erişilir. Dizi ile gerçeklense bile alttaki elemanlara erişilmez; yığın soyut veri tipidir (abstract data type).
- Kuyrukta elemanları her `dequeue` sonrası öne kaydırmak gereksiz yüktür. Bunun yerine işaretçiler ve dairesel kuyruk kullanılır.
- Kuyrukta iki işaretçinin birbirine göre konumu kritiktir. Boş ve dolu durumlarının kontrolü dikkat ister.
- Kuyruk gerçeklemesinde `front` ve `rear`ın başlangıç değeri ve artırma sırası birbiriyle uyumlu olmalıdır.
- Linked list ile ilgili bir ödev ertesi gün online.yildiz.edu.tr üzerinden paylaşılacak.

Kısa Tekrar Notları

- Yığının kullanım alanları:
  - Fonksiyon çağrıları ve özyineleme; aşırı iç içe çağrı stack overflow verir.
  - Geri al / yinele (undo / redo).
  - İfade değerlendirme (expression evaluation).
  - Tarayıcı geçmişi, arama kaydı, bildirimler.
  - Parantez dengesi kontrolü.
  - Geri izleme (backtracking), örneğin labirent çözümü.
- Yığının avantajı: `push` ve `pop` $O(1)$. Dezavantajı: rastgele erişim (random access) yok, taşma (overflow) ve boşalma (underflow) riski.
- Kuyruk, FIFO (first in first out) çalışır. Örnekler: yazıcı kuyruğu, ağ paketleri, işletim sistemi süreçleri.
- Kuyruk türleri:
  - Normal kuyruk.
  - Dairesel kuyruk.
  - Öncelikli kuyruk (priority queue); genelde heap ile gerçeklenir.

  - Çift uçlu kuyruk (double ended queue, deque).
- Kuyruk işlemleri: `isEmpty`, `isFull`, `enqueue`, `dequeue`, `initQueue`.
- Dairesel kuyrukta `rear = (rear + 1) % SIZE` ve `front = (front + 1) % SIZE` ile başa dönülür.

Özellik	Yığın	Kuyruk
Mantık	LIFO	FIFO
Ekleme/silme noktası	Tek uç	Ekleme `rear`, silme `front`
İşaretçi sayısı	1 (`top`)	2 (`front`, `rear`)
İşlemler	`push`, `pop`	`enqueue`, `dequeue`
Türevler	Yok denecek kadar az	Dairesel, öncelikli, deque

Detaylı Açıklamalar

Infix'ten Prefix'e Dönüşüm

Örnek: $A * B + C/D$.

- Prefix karşılığı: `+ * A B / C D`.
- Önce $A * B$, sonra C/D hesaplanır, en son ikisi toplanır.

Algoritma, $A + B * C$ örneği üzerinden:

Adım	İşlem	Sonuç
1	Infix ifadeyi ters çevir	`C * B + A`
2	Postfix'e dönüştür	`C B * A +`
3	Sonucu ters çevir	`+ A * B C`

- Ters çevirme adımı her ifade için gerekir; bu örneğe özel değildir.
- Ters çevirmeden önce postfix'e çevirip sonra ters çevirirseniz doğru prefix elde edilmez.
- Prefix dönüşümünde operatör önceliği karşılaştırmasında küçük bir fark vardır. Postfix'te "eşit veya büyük" önceliğe bakılırken burada yalnız "büyük" durumu dikkate alınır. Hoca ayrıntıyı araştırmaya bıraktı.

Daha karmaşık örnek (tahtadan, ifadenin $A/B \wedge C + D * E - A * C$ olduğu anlaşılıyor):

- Üs alma (`^`) en yüksek önceliklidir. Önce $B \wedge C$ hesaplanır, sonra A buna bölünür.
- Sonra $D * E$, sonra toplama, sonra $A * C$, en son çıkarma yapılır.
- Postfix: `A B C ^ / D E * + A C * -`
- Prefix: `- + / A ^ B C * D E * A C`

Alıştırma olarak verilen ifadeler (prefix ve postfix'leri çıkarılacak, sonraki derste bakılacak):

- $A \wedge B * C - D + E/F/(G + H)$
- $A - B/(C * D \wedge E)$ (ifadenin parantezleri altyazıdan tam anlaşılmıyor)

Ekstra Alan Kullanmadan Yığını Ters Çevirme

- Geçen hafta ikinci bir yığın kullanılmıştı: birinden `pop`, diğerine `push`. Bu yöntem n elemanlık ek alan ister.
- Ek alan olmadan çözüm: yığını bağlı liste (linked list) ile gerçeklemek.
- `head` en üstteki elemanı gösterir. Liste `NULL` görülene kadar dolaşılır.
- Her adımda düğümün bağlantısı ters yöne çevrilir.
- En sonda `head`, eskiden en altta olan düğümün adresine atanır.
- Böylece ek alan harcanmaz; bunun karşılığında işlem süresi artabilir. Ek alan ile zaman arasında bir takas (trade-off) vardır.

Kuyruk Yapısı

- Kuyruğun başı `front`, sonu `rear` ile tutulur.
- `front`, sıradaki hizmet alacak elemanı gösterir.
- `rear`, en son eklenen elemanı gösterir; yeni elemanın yerini belirlemeye yarar.
- `dequeue` sonrası elemanın değeri dizide kalabilir. Önemli olan `front`un ilerlemesidir.

Bağlı liste ile kuyruk:

- `front` listenin başını, `rear` son düğümü tutar.
- `dequeue`: baştaki düğüm alınır, `front` bir sonrakine geçer.
- `enqueue`: yeni düğüm oluşturulur, son düğüme bağlanır, `rear` yeni düğümü gösterir.

Dizi ile Normal Kuyruk

Derste gösterilen fonksiyonların özü:

```
int isEmpty() { return front == -1; }        /* ya da size == 0 */
int isFull()  { return rear == MAX - 1; }

int enqueue(int value) {
    if (isFull()) return 0;
    q[++rear] = value;
    return 1;
}

int dequeue(int *x) {
    if (isEmpty()) return 0;
    *x = q[front++];
    return 1;
}
```

- `front++`: önce değeri kullan, sonra artır. `++front`: önce artır, sonra kullan.
- Hoca bu kodda bir sorun olduğunu gösterdi ve öğrencilere çözdürdü:
  - Başta `front = rear = -1` ise ilk `enqueue` sonrası `rear = 0` olur, `front` -1 kalır.
  - `dequeue`, `q[-1]` gözüne erişmeye çalışır.
  - `++front` önerisi ilk adımı çözer. Ancak bir eleman daha gelince yanlış elemanı çıkarır.
- Önerilen çözümler:
  - `front` ve `rear`ı 0'dan başlatmak. `rear == 0` boş, `rear == MAX` dolu demektir.
  - İlk `enqueue`'da `front`u da 0 yapmak.
  - Ayrı bir `size` sayacı tutmak. Kuyruk tamamen boşalınca `front` ve `rear` başlangıç değerlerine geri alınmalıdır.
- Normal kuyruk alanı boşa harcar. Bu yüzden asıl kullanılan dairesel kuyruktur.

Dairesel Kuyruk

```
void initQueue() { front = -1; rear = -1; }

int isEmpty() { return front == -1; }

int isFull() {
    return (front == rear + 1) || (front == 0 && rear == SIZE - 1);
}

int enqueue(int value) {
    if (isFull()) return 0;
    if (front == -1) front = 0;          /* ilk eleman */
    rear = (rear + 1) % SIZE;
    q[rear] = value;
    return 1;
}
```

```c
int dequeue(int *x) {
    if (isEmpty()) return 0;
    *x = q[front];
    if (front == rear) {                /* son eleman da çıktı */
        front = -1;
        rear = -1;
    } else {
        front = (front + 1) % SIZE;
    }
    return 1;
}
```

Örnek akış (`SIZE = 4`):

İşlem	Kuyruk	`front`	`rear`
Başlangıç	boş	-1	-1
`enqueue(A)`	A	0	0
`enqueue(B), C, D`	A B C D	0	3
`enqueue(E)`	dolu, başarısız	0	3
`dequeue`	B C D	1	3
`enqueue(E)`	E B C D (E başa döndü)	1	0
`enqueue(F)`	dolu, çünkü `front == rear + 1`	1	0

- Doluluk iki durumda olur:
  - `rear`, `front`un hemen arkasına dolanmıştır (`front == rear + 1`).
  - Hiç eleman çıkmadan dizi dolmuştur (`front == 0`, `rear == SIZE - 1`).
- `front == rear` iken `dequeue` yapılırsa son eleman çıkmıştır. Kuyruğun boş anlaşılması için `front` ve `rear` -1'e çekilir.
- Bu kontrol yapılmazsa `isEmpty` kuyruğun boşaldığını yakalayamaz.

İki Yığınla Kuyruk

- `enqueue`: A yığınına `push`.
- `dequeue`:
  - A'daki elemanlar tek tek `pop` edilip B'ye `push` edilir.
  - B'nin tepesindeki eleman (ilk gelen) çıkarılır.
  - Kalanlar tekrar A'ya taşınır.
- Bu kaba (brute force) çözümdür. İki yığını dengeli kullanan daha verimli çözümler de vardır.

Kuyruk ile Yığın

- `push`: kuyruğa `enqueue`.
- `pop`, iki kuyrukla:
  - A'dan son eleman hariç tümü `dequeue` edilip B'ye `enqueue` edilir.
  - A'da kalan son eleman (en son gelen) çıkarılır.
  - Kalanlar A'ya geri taşınır.
- `push` $O(1)$, `pop` $O(n)$ olur.
- Yalnız `enqueue` ve `dequeue` kullanılabilir. `rear`dan eleman çıkarmak kuyruğun tanımını bozar.
- Derste bir öğrenci tek kuyrukla çözüm önerdi: `size - 1` kez `dequeue` edip aynı kuyruğa `enqueue` etmek. Böylece son gelen eleman başa geçer ve `dequeue` ile çıkarılır. Hoca bunu sonraki derste netleştireceğini söyledi.

Ders sonrası notlar

- **Tek kuyrukla yığın:** Öğrencinin önerdiği yöntem doğrudur ve literatürde bilinen bir çözümdür. `pop` için `size - 1` eleman döndürülür, sonra baştaki eleman çıkarılır. Döndürme

sırasında kuyruk dolmaz; her `dequeue` bir yer açar, ardından gelen `enqueue` o yeri kullanır. Maliyet yine $O(n)$'dir.

- **İkinci yığınla ters çevirme:** Hoca bu işlemin "O(1) karmaşıklıkla" yapıldığını söyledi. Doğrusu $O(n)$ zaman ve $O(n)$ ek alandır; her eleman bir kez taşınır.
- **`size` ile doluluk:** Hoca `size` tutan sürümde doluluk koşulunu `size == MAX - 1` olarak söyledi. `size` eleman sayısını tuttuğu için doğru koşul `size == MAX`'tır. Ayrıca `size`, `enqueue` sonrası artırılır, `dequeue` sonrası azaltılır; derste bu bir yerde ters söylendi.
- **Dairesel kuyrukta doluluk:** İki koşul tek ifadeyle de yazılabilir: `(rear + 1) % SIZE == front`.
- **Bağlı listeyi ters çevirme:** Derste "previous" ve "next" sözcükleri karışık kullanıldı. Tek yönlü listede standart yöntem üç işaretçi (`prev`, `cur`, `next`) ile her düğümün `next` alanını bir öncekine çevirmektir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.